Data Preparation for Machine Learning and Deep Learning

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")
import duckdb as db
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler, TargetEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
```

In [14]:
```
df=pd.read_parquet("../data/cleaned/dataset.parquet")
```

In [15]:
```
df.columns
```

Out[15]:
```
Index(['market_id', 'created_at', 'actual_delivery_time', 'store_id',
       'store_primary_category', 'order_protocol', 'total_items', 'subtotal',
       'num_distinct_items', 'min_item_price', 'max_item_price',
       'total_onshift_partners', 'total_busy_partners',
       'total_outstanding_orders', 'created_at_year', 'created_at_month',
       'created_at_day', 'created_at_hour', 'created_at_minute',
       'created_at_second', 'actual_delivery_time_year',
       'actual_delivery_time_month', 'actual_delivery_time_day',
       'actual_delivery_time_hour', 'actual_delivery_time_minute',
       'actual_delivery_time_second', 'created_at_month_name',
       'actual_delivery_time_month_name', 'created_at_day_of_week',
       'actual_delivery_time_day_of_week', 'created_at_day_name',
       'actual_delivery_time_day_name', 'created_at_week_number',
       'actual_delivery_time_week_number', 'created_at_week_of_month',
       'actual_delivery_time_week_of_month', 'delivery_time_seconds',
       'delivery_time_minutes'],
      dtype='object')
```

In [16]:
```
to_drop = [
     'created_at', 'actual_delivery_time', 'store_id',
     'created_at_year', 'created_at_month',
       'created_at_second', 'actual_delivery_time_year',
       'actual_delivery_time_month', 'actual_delivery_time_day',
       'actual_delivery_time_hour', 'actual_delivery_time_minute',
       'actual_delivery_time_second', 'created_at_month_name',
       'actual_delivery_time_month_name',
       'actual_delivery_time_day_of_week', 'created_at_day_name',
       'actual_delivery_time_day_name', 'created_at_week_number',
       'actual_delivery_time_week_number',
       'actual_delivery_time_week_of_month', 'delivery_time_seconds',
]
```

In [17]:
```
df = df.drop(columns=to_drop)
df.columns
```

Out[17]:
```
Index(['market_id', 'store_primary_category', 'order_protocol', 'total_items',
       'subtotal', 'num_distinct_items', 'min_item_price', 'max_item_price',
       'total_onshift_partners', 'total_busy_partners',
       'total_outstanding_orders', 'created_at_day', 'created_at_hour',
       'created_at_minute', 'created_at_day_of_week',
       'created_at_week_of_month', 'delivery_time_minutes'],
      dtype='object')
```

In [18]:
```
df=df.drop_duplicates()
```

In [19]:
```
df.shape
```

Out[19]:
```
(177544, 17)
```

In [20]:
```
df=df[df["total_items"] < 100]
df=df[df["delivery_time_minutes"] < 200]
df=df[df["max_item_price"] < 10000]
df=df[df["min_item_price"] < 10000]
df=df[(df["subtotal"] < 15000) & (df["subtotal"] > 0)]
```

In [21]:
```
df["is_weekend"] = np.where(df["created_at_day_of_week"] >= 5, 1, 0)
df["is_peak_hour"] = np.where(df["created_at_hour"].isin([14,15,20,21]), 1, 0)
```

In [22]:
```
df.shape
```

Out[22]:
```
(177285, 19)
```

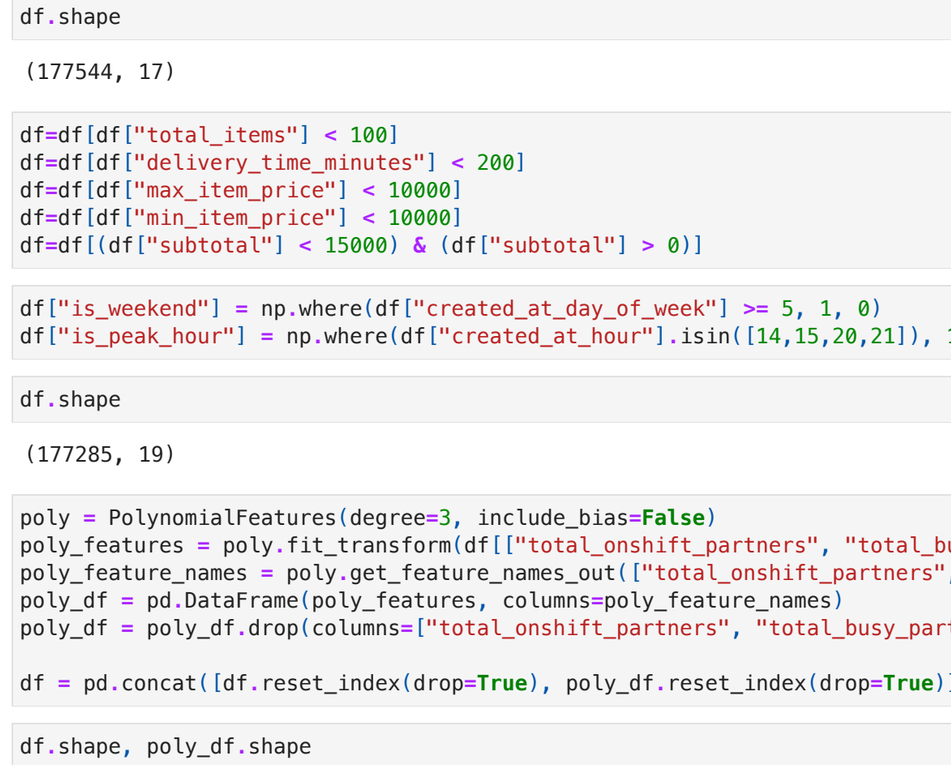
In [23]:
```
poly = PolynomialFeatures(degree=3, include_bias=False)
poly_features = poly.fit_transform(df[["total_onshift_partners", "total_busy_partners", "total_outstanding_orders"]])
poly_feature_names = poly.get_feature_names_out(["total_onshift_partners", "total_busy_partners", "total_outstanding_orders"])
poly_df = pd.DataFrame(poly_features, columns=poly_feature_names)
poly_df = poly_df.drop(columns=["total_onshift_partners", "total_busy_partners", "total_outstanding_orders"])

df = pd.concat([df.reset_index(drop=True), poly_df.reset_index(drop=True)], axis=1)
```

In [24]:
```
df.shape, poly_df.shape
```

Out[24]:
```
((177285, 35), (177285, 16))
```

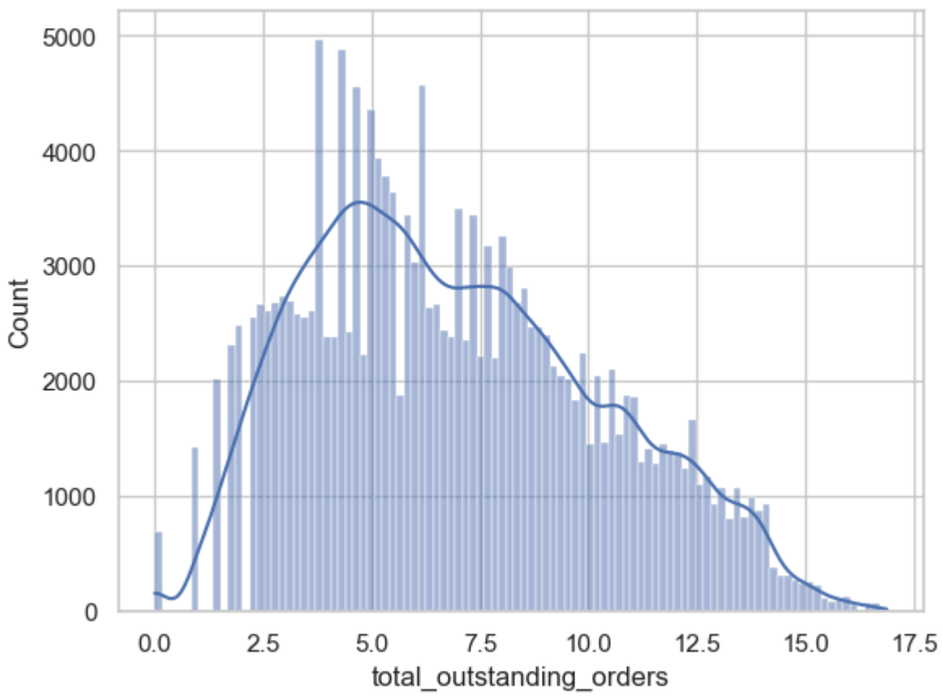
In [25]:
```
sns.histplot(data=poly_df, x="total_onshift_partners total_busy_partners", bins=50)
```

Out[25]:
```
<Axes: xlabel='total_onshift_partners total_busy_partners', ylabel='Count'>
```



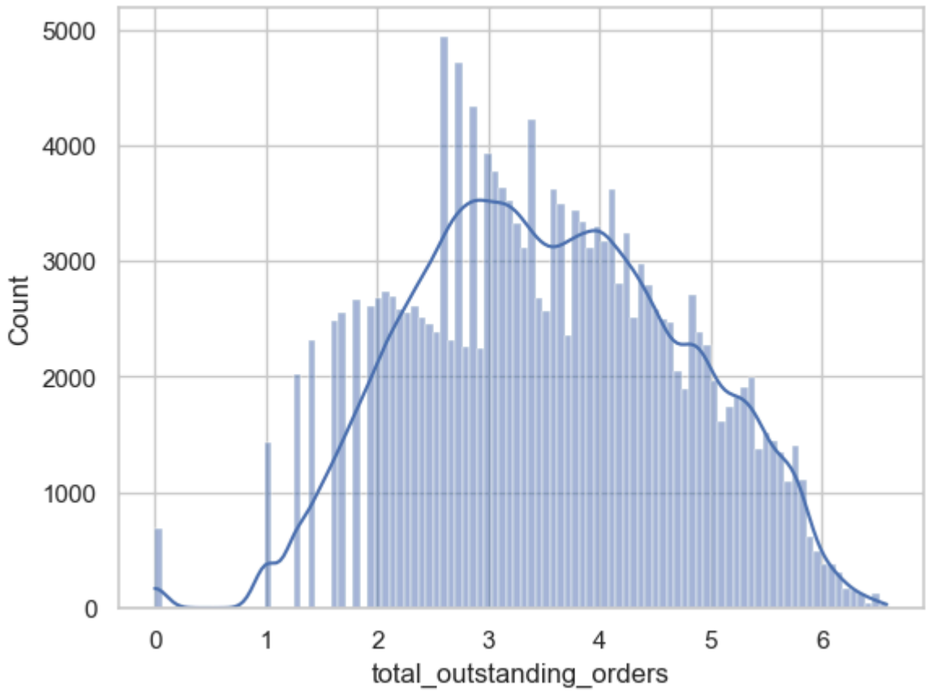
In [26]:
```
# sns.histplot(np.log1p(df["delivery_time_minutes"]), bins=100, kde=True)
# cube root
sns.histplot(np.sqrt(df["total_outstanding_orders"]), bins=100, kde=True)
```

Out[26]: <Axes: xlabel='total_outstanding_orders', ylabel='Count'>



In [27]:
```
sns.histplot(np.cbrt(df["total_outstanding_orders"]), bins=100, kde=True)
```

Out[27]: <Axes: xlabel='total_outstanding_orders', ylabel='Count'>

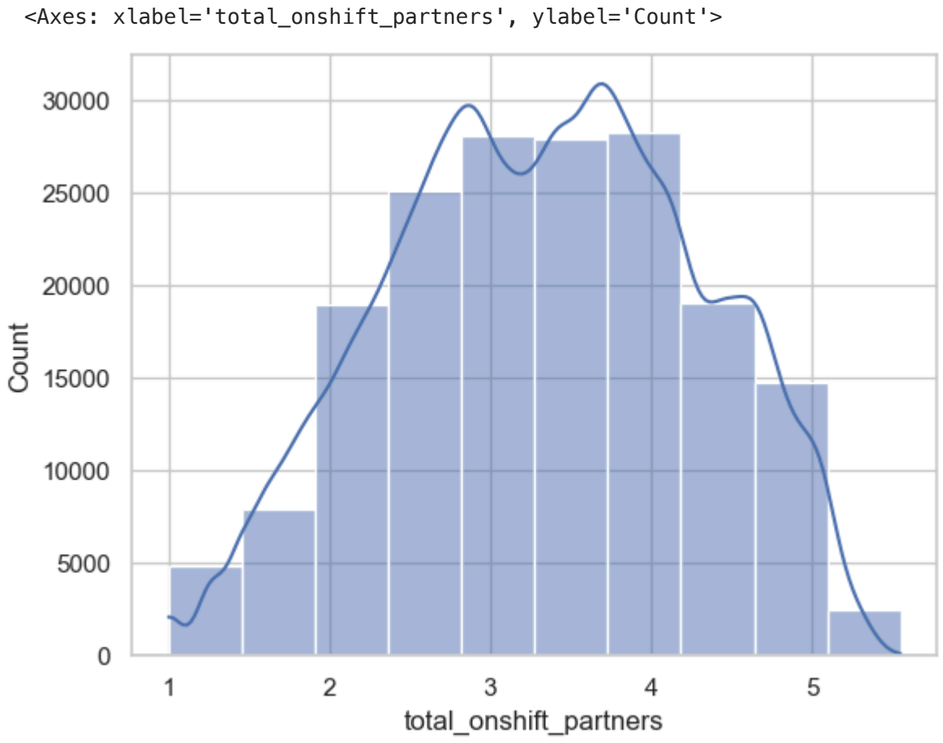


In [28]:
```
df["num_distinct_items"].min(), df["subtotal"].max()
```

Out[28]: (1, 14955)

In [29]:
```
sns.histplot(x=np.cbrt(df["total_onshift_partners"]), bins=10, kde=True)
```

Out[29]: <Axes: xlabel='total_onshift_partners', ylabel='Count'>



In [30]:
```
# df["total_items"] = np.log(df["total_items"])
# df["max_item_price"] = np.cbrt(df["max_item_price"])
# df["min_item_price"] = np.cbrt(df["min_item_price"])
# df["subtotal"] = np.cbrt(df["subtotal"])
# df["total_onshift_partners"] = np.cbrt(df["total_onshift_partners"])
# df["total_busy_partners"] = np.cbrt(df["total_busy_partners"])
# df["total_outstanding_orders"] = np.cbrt(df["total_outstanding_orders"])
```

In [31]:
```
df.shape
```

Out[31]: (177285, 35)

In [32]:
```
df
```

Out[32]:

	market_id	store_primary_category	order_protocol	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	...	total_onshift_partners^3
0	1.0	american	1	4	3441	4	557.0	1239.0	33.0	14.0	...	35937.0
1	2.0	mexican	2	1	1900	1	1400.0	1400.0	1.0	2.0	...	1.0
2	3.0	other	1	1	1900	1	1900.0	1900.0	1.0	0.0	...	1.0
3	3.0	other	1	6	6900	5	600.0	1800.0	1.0	1.0	...	1.0
4	3.0	other	1	3	3900	3	1100.0	1600.0	6.0	6.0	...	216.0
...	...	...	...	...	...	...	...	...	...	...	...	...
177280	1.0	fast	4	3	1389	3	345.0	649.0	17.0	17.0	...	4913.0
177281	1.0	fast	4	6	3010	4	405.0	825.0	12.0	11.0	...	1728.0
177282	1.0	fast	4	5	1836	3	300.0	399.0	39.0	41.0	...	59319.0
177283	1.0	sandwich	1	1	1175	1	535.0	535.0	7.0	7.0	...	343.0
177284	1.0	sandwich	1	4	2605	4	425.0	750.0	20.0	20.0	...	8000.0

177285 rows × 35 columns

In [33]:
```
plt.figure(figsize=(20, 20))
sns.heatmap(df.drop(['market_id', 'store_primary_category', 'order_protocol'], axis=1).corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Heatmap")
```

Out[33]: Text(0.5, 1.0, 'Correlation Heatmap')

In [34]:
```
df.columns
```

Out[34]:
```
Index(['market_id', 'store_primary_category', 'order_protocol', 'total_items',
       'subtotal', 'num_distinct_items', 'min_item_price', 'max_item_price',
       'total_onshift_partners', 'total_busy_partners',
       'total_outstanding_orders', 'created_at_day', 'created_at_hour',
       'created_at_minute', 'created_at_day_of_week',
       'created_at_week_of_month', 'delivery_time_minutes', 'is_weekend',
       'is_peak_hour', 'total_onshift_partners^2',
       'total_onshift_partners total_busy_partners',
       'total_onshift_partners total_outstanding_orders',
       'total_busy_partners^2', 'total_busy_partners total_outstanding_orders',
       'total_outstanding_orders^2', 'total_onshift_partners^3',
       'total_onshift_partners^2 total_busy_partners',
       'total_onshift_partners^2 total_outstanding_orders',
       'total_onshift_partners total_busy_partners^2',
       'total_onshift_partners total_busy_partners total_outstanding_orders',
       'total_onshift_partners total_outstanding_orders^2',
       'total_busy_partners^3',
       'total_busy_partners^2 total_outstanding_orders',
       'total_busy_partners total_outstanding_orders^2',
       'total_outstanding_orders^3'],
      dtype='object')
```

In [35]:
```
df[["created_at_day", "created_at_week_of_month"]].sample(20)
```

Out[35]:

	created_at_day	created_at_week_of_month
65721	25	4
90122	12	2
146653	17	3
139955	25	4
160201	5	1
54085	15	3
73879	1	1
46481	2	1
175668	8	2
16242	22	4
49207	5	1
144666	28	4
71066	11	2
66843	13	2
173801	22	4
70359	30	5
173948	11	2
87911	8	2
126283	5	1
106092	17	3

In [36]:
```
cat_df = df["store_primary_category"].value_counts().reset_index().rename(columns={ 'store_primary_category': 'category'})
cat_df
```

Out[36]:

	category	count
0	american	18076
1	pizza	15684
2	mexican	15599
3	burger	9833
4	sandwich	8984
...	...	...
68	african	10
69	lebanese	9
70	belgian	2
71	chocolate	1
72	alcohol-plus-food	1

73 rows × 2 columns

In [37]:
```
df=df.merge(cat_df, left_on="store_primary_category", right_on="category", how="left")
```

In [38]:
```
df["store_primary_category"] = np.where(df["count"] < 20, "other", df["store_primary_category"])
```

In [39]:
```
df=df.drop(columns=["category", "count"])
```

In [40]:
```
df.columns
```

Out[40]:
```
Index(['market_id', 'store_primary_category', 'order_protocol', 'total_items',
       'subtotal', 'num_distinct_items', 'min_item_price', 'max_item_price',
       'total_onshift_partners', 'total_busy_partners',
       'total_outstanding_orders', 'created_at_day', 'created_at_hour',
       'created_at_minute', 'created_at_day_of_week',
       'created_at_week_of_month', 'delivery_time_minutes', 'is_weekend',
       'is_peak_hour', 'total_onshift_partners^2',
       'total_onshift_partners total_busy_partners',
       'total_onshift_partners total_outstanding_orders',
       'total_busy_partners^2', 'total_busy_partners total_outstanding_orders',
       'total_outstanding_orders^2', 'total_onshift_partners^3',
       'total_onshift_partners^2 total_busy_partners',
       'total_onshift_partners^2 total_outstanding_orders',
       'total_onshift_partners total_busy_partners^2',
       'total_onshift_partners total_busy_partners total_outstanding_orders',
       'total_onshift_partners total_outstanding_orders^2',
       'total_busy_partners^3',
       'total_busy_partners^2 total_outstanding_orders',
       'total_busy_partners total_outstanding_orders^2',
       'total_outstanding_orders^3'],
      dtype='object')
```

In [41]:
```
all_cols=['market_id', 'store_primary_category', 'order_protocol', 'total_items',
       'subtotal', 'num_distinct_items', 'min_item_price', 'max_item_price',
       'total_onshift_partners', 'total_busy_partners',
       'total_outstanding_orders', 'created_at_day', 'created_at_hour',
       'created_at_minute', 'created_at_day_of_week',
       'created_at_week_of_month', 'delivery_time_minutes',  'is_weekend',
       'is_peak_hour', 'total_onshift_partners^2',
       'total_onshift_partners total_busy_partners',
       'total_onshift_partners total_outstanding_orders',
       'total_busy_partners^2', 'total_busy_partners total_outstanding_orders',
       'total_outstanding_orders^2', 'total_onshift_partners^3',
       'total_onshift_partners^2 total_busy_partners',
```

```
        'total_onshift_partners^2 total_outstanding_orders',
        'total_onshift_partners total_busy_partners^2',
        'total_onshift_partners total_busy_partners total_outstanding_orders',
        'total_onshift_partners total_outstanding_orders^2',
        'total_busy_partners^3',
        'total_busy_partners^2 total_outstanding_orders',
        'total_busy_partners total_outstanding_orders^2',
        'total_outstanding_orders^3']

date_time_cols = [ 'created_at_hour', 'created_at_minute',
    'created_at_day_of_week'
]

def convert_cyclical(df, col, max_val):
    df[col + '_sin'] = np.sin(2 * np.pi * df[col] / max_val).round(8)
    df[col + '_cos'] = np.cos(2 * np.pi * df[col] / max_val).round(8)
    return df
```

In [42]:
```
for col in date_time_cols:
    df = convert_cyclical(df, col, df[col].max())

df = df.drop(columns=date_time_cols)
```

In [43]:
```
category_label_encoder = LabelEncoder()
df['store_primary_category_le'] = category_label_encoder.fit_transform(df['store_primary_category'])
```

In [44]:
```
order_protocol_onehot_encoder = OneHotEncoder(sparse_output=False)
order_protocol_onehot = order_protocol_onehot_encoder.fit_transform(df[['order_protocol']])
order_protocol_onehot_df = pd.DataFrame(order_protocol_onehot, columns=order_protocol_onehot_encoder.get_feature_names_out(['order_protocol']))
df = pd.concat([df, order_protocol_onehot_df], axis=1)
df = df.drop(columns=['order_protocol'])
```

```
market_id_onehot_encoder = OneHotEncoder(sparse_output=False)
market_id_onehot = market_id_onehot_encoder.fit_transform(df[['market_id']])
market_id_onehot_df = pd.DataFrame(market_id_onehot, columns=market_id_onehot_encoder.get_feature_names_out(['market_id']))
df = pd.concat([df, market_id_onehot_df], axis=1)
df = df.drop(columns=['market_id'])
```

In [45]:
```
df
```

Out[45]:

	store_primary_category	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders	created_at_day	...	order_protoc
0	american	4	3441	4	557.0	1239.0	33.0	14.0	21.0	6	...	
1	mexican	1	1900	1	1400.0	1400.0	1.0	2.0	2.0	10	...	
2	other	1	1900	1	1900.0	1900.0	1.0	0.0	0.0	22	...	
3	other	6	6900	5	600.0	1800.0	1.0	1.0	2.0	3	...	
4	other	3	3900	3	1100.0	1600.0	6.0	6.0	9.0	14	...	
...	...	...	...	...	...	...	...	...	...	...	...	
177280	fast	3	1389	3	345.0	649.0	17.0	17.0	23.0	16	...	
177281	fast	6	3010	4	405.0	825.0	12.0	11.0	14.0	12	...	
177282	fast	5	1836	3	300.0	399.0	39.0	41.0	40.0	23	...	
177283	sandwich	1	1175	1	535.0	535.0	7.0	7.0	12.0	1	...	
177284	sandwich	4	2605	4	425.0	750.0	20.0	20.0	23.0	8	...	

177285 rows × 50 columns

In [46]:
```
X=df.drop(columns=['delivery_time_minutes'])
y=np.cbrt(df["delivery_time_minutes"])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.5, random_state=42)
X_train.shape, X_val.shape, X_test.shape, y_train.shape, y_val.shape, y_test.shape
```

Out[46]:
```
((124099, 49), (26593, 49), (26593, 49), (124099,), (26593,), (26593,))
```

In [47]:
```
y_train.to_frame().isna().sum()
```

Out[47]:
```
delivery_time_minutes    0
dtype: int64
```

In [48]:
```
category_target_encoder = TargetEncoder(target_type="continuous")
X_train["store_primary_category"]=category_target_encoder.fit_transform(X_train[['store_primary_category']], y_train)
X_val["store_primary_category"]=category_target_encoder.transform(X_val[['store_primary_category']])
X_test["store_primary_category"]=category_target_encoder.transform(X_test[['store_primary_category']])
```

In [49]:
```
X_train_store_category_le = X_train.pop('store_primary_category_le')
X_val_store_category_le = X_val.pop('store_primary_category_le')
X_test_store_category_le = X_test.pop('store_primary_category_le')
```

In [50]:
```
X_train.shape
```

Out[50]:
```
(124099, 48)
```

In [51]:
```
y.shape
```

Out[51]:
```
(177285,)
```

In [52]:
```
cols=X_train.columns
```

In [53]:
```
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_val = sc.transform(X_val)
X_test = sc.transform(X_test)
```

In [54]:
```
sc_y = StandardScaler()
y_train = sc_y.fit_transform(y_train.values.reshape(-1, 1))
y_val = sc_y.transform(y_val.values.reshape(-1, 1))
y_test = sc_y.transform(y_test.values.reshape(-1, 1))
```

In [55]:
```
X_train.shape
```

Out[55]:
```
(124099, 48)
```

In [56]:
```
pd.Series(y_train.reshape(-1))
```

Out[56]:
```
0        -0.723537
1        -0.510422
2        -1.108517
3         0.410726
4         1.026274
            ...
124094   -1.028169
124095    0.830366
124096    1.568953
124097   -1.449784
124098   -0.723537
Length: 124099, dtype: float64
```

In [57]:
```
X_train = pd.DataFrame(X_train, columns=cols)
X_val = pd.DataFrame(X_val, columns=cols)
X_test = pd.DataFrame(X_test, columns=cols)
```

```
y_train = pd.Series(y_train.reshape(-1), name="delivery_time_minutes")
y_val = pd.Series(y_val.reshape(-1), name="delivery_time_minutes")
y_test = pd.Series(y_test.reshape(-1), name="delivery_time_minutes")
```

In [58]:
```
import pickle

with open('../data/processed/dataset.pkl', 'wb') as f:
    pickle.dump({'X_train': X_train,
                 'y_train': y_train,
                 'X_val': X_val,
                 'y_val': y_val,
                 'X_test': X_test,
                 'y_test': y_test,
                 'X_train_store_category_le': X_train_store_category_le,
                 'X_val_store_category_le': X_val_store_category_le,
                 'X_test_store_category_le': X_test_store_category_le,
                "y_scaler": sc_y,
                "X_scaler": sc,
                },
                f)
```

In [59]:
```
with open('../data/processed/dataset.pkl', 'rb') as f:
    dataset = pickle.load(f)
```

In [60]:
```
dataset.keys()
```

Out[60]:
```
dict_keys(['X_train', 'y_train', 'X_val', 'y_val', 'X_test', 'y_test', 'X_train_store_category_le', 'X_val_store_category_le', 'X_test_store_category_le', 'y_scaler', 'X_scaler'])
```

In [61]:
```
df.to_parquet("../data/processed/dataset.parquet", index=False)
```

In [62]:
```
X_train
```

Out[62]:

	store_primary_category	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders	created_at_day	...	order_prc
0	0.139958	-0.887847	-0.774580	-1.034172	1.175252	0.233244	0.356917	-0.239323	-0.063258	-1.134736	...	-0
1	-0.294285	0.323576	0.936110	0.203869	-0.179647	0.466436	-0.898395	-0.866322	-0.691474	0.050511	...	-0
2	0.082312	0.727384	0.600475	0.822890	-0.664373	0.053865	-0.985975	-1.023072	-0.938953	0.912508	...	-0
3	-1.741454	-0.080232	-1.258467	0.203869	-1.151047	-1.817052	-0.927588	-1.023072	-0.938953	0.697009	...	3
4	0.719151	1.938807	1.653122	2.060932	-1.053712	-0.295923	1.115942	0.857927	1.078952	1.666756	...	-0
...	...	...	...	...	...	...	...	...	...	...	...	
124094	-1.667240	-0.080232	-0.288488	0.203869	-0.179647	-0.835852	-0.256143	-0.803622	-0.653400	1.451257	...	-0
124095	0.304998	-0.484040	-0.667111	-0.415151	0.007236	-0.663649	1.641422	1.798427	2.620935	1.666756	...	-0
124096	0.346322	0.727384	1.802477	1.441911	-0.664373	1.348978	0.619656	0.701177	0.679178	1.235757	...	-0
124097	-1.359593	-0.080232	-0.443905	0.203869	-0.576772	-0.609835	-0.139369	-0.490123	-0.577253	0.589259	...	-0
124098	-1.285614	-0.484040	-0.309982	-0.415151	0.735299	-0.172152	-1.190328	-1.148472	-0.977026	1.235757	...	-0

124099 rows × 48 columns

In [63]:
```
y_train
```

Out[63]:
```
0        -0.723537
1        -0.510422
2        -1.108517
3         0.410726
4         1.026274
            ...
124094   -1.028169
124095    0.830366
124096    1.568953
124097   -1.449784
124098   -0.723537
Name: delivery_time_minutes, Length: 124099, dtype: float64
```